And then, there is a ^C (Ctrl-C) immediately after that, which basically terminates the formatting. Aha! Did Pugs realise there was something important on the pen drive? Not really, as his pen drive is already empty. Actually, what happened is that formatting was going on for quite some time—so Pugs had some doubts about his code changes and terminated the run. Reviewing his code didn't yield much, so he reissued the format command, this time with the *time* command, to figure out exactly how much time the formatting takes and then maybe debug or fix that. And finally... the formatting completed after a whopping 430.88 seconds (7+ *minutes!*). *time* basically shows the real time taken (including the time when other processes have been running after the context switch), the time executed in user space, and the time executed in kernel space. That's a lot—something obviously needs optimisation. And it didn't take much time for a close review to determine the issue. The code at fault is the *clear_file_entries()* function, which clears file entries by writing 64-byte-sized file entries one by one—which is pretty non-optimal. A better approach would be to fill up a block with such entries, and then write these blocks one by one. In case of a 512-byte block (i.e., *SIMULA_FS_BLOCK_SIZE* defined as 512), that would mean 8 file entries in a 512-byte block, and then writing these 512-byte blocks, one by one. So, Pugs changed the *clear_file_entries()* function to do that, and voilà! Formatting completed in a little less than 26 seconds. Here's the re-written *clear_file_entries()* function:

```
void clear_file_entries(int sfs_handle, sfs_super_block_t *sb)
{
    int i;
    uint1_t block[SIMULA_FS_BLOCK_SIZE];

    for (i = 0; i < sb->block_size / sb->entry_size; i++)
    {
        memcpy(block + i * sb->entry_size, &fe, sizeof(fe));
    }
    for (i = 0; i < sb->entry_table_size; i++)
    {
        write(sfs_handle, block, sizeof(block));
    }
}
```

Now, you may plug out and plug in the pen drive again. It is neither auto-mounted, nor are you able to mount it. That's expected—it has been formatted with a file system for which there is no (kernel) module to decode. So, coding would be the ultimate step to get everything working like with any other existing file system (*vfat*, *ext3*, …). To get your pen drive usable again, re-format it with the FAT32 (*vfat*) file system (as the root, or with *sudo*), as follows:

```
# mkfs.vfat /dev/sdb1 # Be careful with the correct partition
device file
```

Then, unplug and reinsert it to get it auto-mounted. But Pugs, being a cool carefree guy, instead went ahead to try browsing the Simula file system created on the pen drive partition.

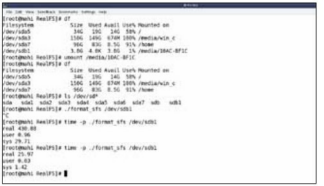


Figure 1: Formatting the pen drive



Figure 2: The SFS browser on the pen drive

Browsing the pen drive partition

Obviously, there were slight modifications to the *browse_sfs.c* application as well, in line with the changes to *format_sfs.c*. The major one was to compulsorily take the partition device file to browse as a command-line argument, instead of browsing the default regular file *.sfsf*.

All the updated files (*sfs_ds.h*, *format_sfs.c*, *browse_sfs.c* and *Makefile*) are available at *http://linuxforu.com/article_source_code/aug12/rfs_code.tar.bz2*.

Figure 2 shows the browser in action. However, the coolest browsing would be done the same way as with all other file systems, using the shell commands *cd*, *ls*, … Yes, and for that, we would need the SFS module in place. Keep following what Pugs is up to, to get that in place.

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medallist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at *http://profession.sarika-pugs.com/*, including information about his open source hardware freak-outs and his various Linux related training sessions and workshops. He can be reached at *email@sarika-pugs.com*.